Object-Oriented Programming
Semester 2025-III
Workshop No. 3

Ronaldo Andres Alvarado Doria – 20251020122
Juliana Valentina Reyes Moreno – 20251020109

Workshop 3 – Application of SOLID Principles

Introduction

In this phase, the project was refined through the integration of the SOLID principles to improve its structure, organization, and scalability. The process included reviewing and updating requirements and design, enhancing UML diagrams, and restructuring the implementation to align with good object-oriented practices. Short code examples and documentation were added to demonstrate how each principle was applied, resulting in a cleaner, more maintainable, and extensible system.

CONCEPTUAL DESIGN UPDATES

Functional Requirements

- **Emotional and Habit Tracking**
  ○ Users can record their daily mood using a scale or emoji. The app shows moods in a color-coded view with simple visual indicators.
  ○ Each habit (such as Sleep, Food, or Journal) is now represented by an independent class that evaluates its own score and performance based on user input.
  ○ New habits can be added later without changing existing logic, since all habits share a common structure through the `Registers` class and the `IAnalyzer` interface.
  (Improved by applying Single Responsibility and Open/Closed principles. The logic became modular, extendable, and easier to maintain.)
- **Reminders**
  ○ Users can receive reminders to record their mood or write in the journal.
  ○ Notifications are now simplified to Push Notifications only, managed by a single

service.
(Reduced complexity while keeping flexibility through the use of abstract classes and dependency inversion.)

- **Suggested Actions**
  ○ The app provides motivational recommendations, such as short walks or relaxation activities.
  ○ Suggestions are now managed through an interface (`SuggestionStrategy`), allowing new recommendation types to be added easily.
  (Refactored to support future expansion without modifying existing code, aligning with the Open/Closed and Interface Segregation principles.)
- **Statistics**
  ○ The app shows simple summaries of emotional progress and habit patterns.
  ○ The statistics feature now uses an abstract base class (`Statistics`) with specialized types such as `WeeklyStatistics` and `MonthlyStatistics`.
  (Supports future data views and extensions while maintaining clean inheritance and consistency with the Liskov Substitution principle.)

Non-Functional Requirements

- **Maintainability**
  ○ The codebase is now fully modular and structured according to SOLID principles.
  ○ Each class has a defined responsibility, and new features can be implemented through interfaces or inheritance without altering existing classes.
  (Major improvement: enhanced readability, easier debugging, and simpler future updates.)
- **Security and Privacy**
  ○ Authentication and user data management were separated into distinct responsibilities (e.g., `UserAuth` and `UserProfile`), ensuring clearer data handling.
  (Improved internal organization and reduced coupling between authentication and personal data.)
- **Reliability**
  ○ Record management was centralized in `Registers` and `DailyRecord`, improving consistency in how data is created, saved, and retrieved.
  (Simpler to control data flow and prevent errors, indirectly increasing reliability.)

Enhanced UML Diagrams

To ensure full SOLID compliance, several structural modifications were introduced into the class design, focusing on the creation of interfaces, abstract classes, and a clear inheritance

hierarchy that made the system more modular and coherent. These updates not only improved organization but also strengthened how data and actions flow throughout the application.

The first major change was the creation of the IAnalyzer interface, which is implemented by all habit-related classes (Sleep, Food, Emotions, and Journal). This interface defines a common structure for evaluating habits through methods like calculateScore() and getColorByScore(). This ensures that each habit follows the same analytical process, demonstrating Interface Segregation and Liskov Substitution, since any habit can replace another within the same flow without breaking functionality.

Similarly, the SuggestionStrategy interface was introduced to manage recommendations dynamically. Implementations such as MoodSuggestion handle specific behaviors while allowing new strategies (e.g., SleepSuggestion or NutritionSuggestion) to be added later without altering existing code. This directly applies the Open/Closed Principle, as the system can grow through extensions rather than modifications.

Two abstract classes, NotificationService and Statistics, were also incorporated. NotificationService defines the general structure for sending notifications, specialized by classes like PushNotificationService. Statistics acts as a base for reporting classes such as WeeklyStatistics and MonthlyStatistics. This abstraction enforces Dependency Inversion, allowing high-level modules to depend on stable contracts rather than concrete implementations.

The inheritance model was refined as well: Registers became the base class for all habit records, containing only shared attributes such as recordDate. Subclasses like Sleep or Journal handle their specific data and analysis logic, ensuring Single Responsibility and improving code reuse and readability.

These structural adjustments significantly enhanced the system's transaction flow—the way information moves between user actions, processing, and output. Each class now handles a single, precise function, allowing data to move logically from recording, to analysis, to result visualization. The use of abstractions and dependency injection ensures that new behaviors can be integrated smoothly, without disrupting existing operations.

As a result, the flow of information through the system became more organized, consistent, and adaptable. Transactions such as recording moods, generating feedback, or sending notifications now occur through a clean sequence of specialized modules. This design reduces coupling, simplifies debugging, and ensures that future extensions can be added with minimal effort while maintaining the stability of the overall system.

(Sequence diagram at the end of the file)

SOLID-Focused Implementation

---

1. Single Responsibility Principle (SRP)

"A class should have only one reason to change."

This principle was applied by **splitting and reorganizing classes that had multiple roles**. Initially, the system had large, multi-purpose classes handling user data, authentication, and statistics all together.

Key Changes:

- The original `User` class was **divided into three focused classes**:
  - `UserProfile` – manages only user information.
  - `UserAuth` – handles authentication and session control.
  - `UserStatisticsAccess` – deals with statistics visualization and suggestions.

This prevents a change in one concern (for example, authentication) from affecting unrelated areas like data or reports.

Representative example:

```
public class UserProfile {
    private String userId, userName, email;
    private int age;
    // Handles only personal information, no logic beyond this.
}

public class UserAuth {
    public void logIn(String userId) { ... }
    public void logOut(String userId) { ... }
    // Independent from profile and statistics.
}
```

- Responsibilities for tracking and evaluating habits were also separated:
  - `Registers` manages base information and date handling.
  - Each habit (`Sleep`, `Food`, `Emotions`, `Journal`) extends `Registers` and adds only its specific evaluation logic.

- `DailyRecord` is responsible only for storing records, not analyzing them.

This makes classes shorter, more focused, and easier to maintain.

Result:

Every class in the system now has **a single, well-defined purpose**:

- `Registers`: base register management.
- `Sleep`, `Food`, `Emotions`, `Journal`: specific habit evaluation.
- `PushNotificationService`: sending notifications.
- `UserProfile`, `UserAuth`, `UserStatisticsAccess`: separate handling of user information, authentication, and data access.

The result is a **modular, purpose-specific code structure**, easier to understand, test, and evolve.

---

2. Open/Closed Principle (OCP)

"Software entities should be open for extension but closed for modification."

The project follows this principle by allowing **new features to be added without modifying existing code**, primarily through the use of inheritance and interfaces.

Key Changes:

- Creation of the **`IAnalyzer`** interface, defining a generic contract for all habit types:

```
public interface IAnalyzer {
    float calculateScore();
    String getColorByScore();
}
```

  New habits (e.g., `Exercise`, `WaterIntake`) can be added by implementing `IAnalyzer`
  without changing the logic of existing classes.

- Abstract base classes such as `Statistics` and `NotificationService` allow easy extension:

- `WeeklyStatistics` and `MonthlyStatistics` extend `Statistics` to provide different summaries.
- `PushNotificationService` extends `NotificationService`, and additional types (e.g., email or SMS) can be added later.

Result:

The system can evolve **by adding new implementations** instead of editing old code.
The main modules, such as the score aggregator or statistics manager, remain untouched.
This demonstrates a complete application of OCP:
existing code remains stable, while the system remains flexible for growth.

---

3. Liskov Substitution Principle (LSP)

"Subclasses should be substitutable for their base classes without altering program correctness."

This principle ensures that **any subclass or implementation can replace its parent** without breaking functionality or logic.

Implementation:

- All habit classes (`Sleep`, `Food`, `Emotions`, `Journal`) **extend `Registers`** and **implement `IAnalyzer`**.
  They can be used interchangeably in the program without additional handling.

For example, in the score aggregator:

```
public class ScoreAggregator {
    private List<IAnalyzer> analyzers;

    public float getAverageScore() {
        float total = 0f;
        for (IAnalyzer a : analyzers) total += a.calculateScore();
        return total / analyzers.size();
    }
}
```

Whether the object is `Sleep`, `Food`, or `Emotions`,
as long as it implements `IAnalyzer`, the system behaves consistently.

- The same applies to abstract hierarchies such as `Statistics` and `NotificationService`.
  A `WeeklyStatistics` instance can be replaced by a `MonthlyStatistics` one, and `PushNotificationService` can replace any other future subclass of `NotificationService`
  without requiring code changes in higher-level components.

Result:

The system maintains true polymorphism.
All subclasses:

- Respect the contract of their parent type.
- Behave consistently when substituted.
- Can be used interchangeably, ensuring code stability and flexibility.

---

4. Interface Segregation Principle (ISP)

"No class should be forced to implement methods it does not use."

This principle was enforced by **splitting large, generic interfaces into smaller, specialized ones**,
so each class implements only what it truly requires.

Key Changes:

- Removed large "multi-purpose" interfaces that mixed unrelated behaviors.
- Introduced **focused, single-purpose interfaces**:
  - `IAnalyzer` – for calculating habit scores.
  - `SuggestionStrategy` – for generating advice or recommendations.
  - `NotificationService` – for sending messages.
  - `Statistics` – for summarizing data.

Each of these interfaces contains only the methods directly related to its role.

Example:

- Habit classes (`Sleep`, `Food`, `Journal`) implement only `IAnalyzer`.
- Suggestion providers (e.g., `MoodSuggestion`) implement `SuggestionStrategy`.

- Notification classes (`PushNotificationService`) extend only `NotificationService`.
- Reporting classes (`WeeklyStatistics`) extend only `Statistics`.

Result:

Classes implement **only what they actually need**.
No class contains unnecessary or empty methods, and the design remains clean, focused, and independent.
This separation also allows new functionality to be added without side effects or tight coupling.

---

5. Dependency Inversion Principle (DIP)

"High-level modules should not depend on low-level modules. Both should depend on abstractions."

This principle was fundamental to making the system **flexible, decoupled, and testable**. Dependencies are injected into classes rather than being created inside them, and all depend on interfaces or abstract types.

Key Changes:

- High-level classes such as `ScoreAggregator` and `UserStatisticsAccess` now depend **only on abstractions**:
  - `ScoreAggregator` uses `List<IAnalyzer>` instead of specific habit classes.
  - `UserStatisticsAccess` depends on `Statistics` and `SuggestionStrategy`,
    both passed through its constructor.

Representative example:

```
NotificationService notifier = new PushNotificationService("dev_001");
notifier.sendNotification("u01", "Your daily summary is ready!");
```

`UserStatisticsAccess` does not know the concrete implementation; it only uses the abstract contracts.

Similarly, in the main class:

```
UserStatisticsAccess userStats = new UserStatisticsAccess(
    new WeeklyStatistics(),
    new MoodSuggestion()
);
```

The system depends on the abstraction `NotificationService`, not the concrete push implementation.

Result:

- High-level components do not create or manage concrete instances.
- Dependencies are injected and can be easily replaced or extended.
- The overall system is decoupled, easier to maintain, and aligned with proper architectural layering.

The dependency flow now moves **from abstractions to implementations**, ensuring strong modularity and compliance with DIP.

---

6. Summary of SOLID Principles in the Final Design

Principle	How It Was Applied
SRP	Large, multi-role classes were reorganized into smaller, single-purpose ones (e.g., `User`, `Registers`).
OCP	The system can grow through extension (`implements`, `extends`) without modifying existing code.
LSP	All subclasses and implementations can safely replace their parent types without changing behavior.
ISP	Interfaces were minimized and specialized, ensuring each class implements only what it needs.

Principle	How It Was Applied
DIP	High-level classes depend on abstractions (`IAnalyzer`, `Statistics`, `SuggestionStrategy`, `NotificationService`), not concrete implementations.

Work in Progress Code & Documentation

This section presents short, representative code snippets and commentary that illustrate how the class structure was refined according to SOLID principles.
Each example highlights an updated class design, its role in the inheritance hierarchy, and the rationale for the change.

---

1. `Registers` – Base Class for Habit Records

```java
public class Registers {
    protected Date recordDate;

    public Registers(Date recordDate) {
        this.recordDate = recordDate;
    }

    public void edit() {
        System.out.println("Editing register...");
    }

    public void cancel() {
        System.out.println("Register cancelled.");
    }
}
```

Rationale:
`Registers` was introduced as a **base abstraction** for all habit entries, encapsulating only the attributes and actions common to any recorded event (date, edit, cancel).
It demonstrates **SRP**, because it manages record operations only, and **OCP**, because all specific habits can extend it without changing its internal logic.
Any new habit class can inherit from `Registers`, gaining these shared capabilities automatically.

---

2. `Sleep` – Specific Habit with Analytical Behavior

```java
public class Sleep extends Registers implements IAnalyzer {
    private int hoursSleep;

    public Sleep(Date recordDate, int hoursSleep) {
        super(recordDate);
        this.hoursSleep = hoursSleep;
    }

    @Override
    public float calculateScore() {
        if (hoursSleep >= 7 && hoursSleep <= 9) return 1f;
        if (hoursSleep >= 5) return 0.6f;
        return 0.3f;
    }

    @Override
    public String getColorByScore() {
        float s = calculateScore();
        return (s >= 0.8f) ? "Green" : (s >= 0.5f ? "Yellow" : "Red");
    }
}
```

Rationale:
`Sleep` inherits from `Registers` to reuse date and record management, and implements `IAnalyzer` to provide its own evaluation logic.
This structure applies **SRP** (only analyzes sleep), **OCP** (can be extended without altering

parent code), and **LSP** (can replace any other `IAnalyzer` object).
The scoring logic is self-contained, so the class can be used interchangeably with other analyzers like `Food` or `Emotions`.

---

3. `Journal` – Custom Habit Implementation

```java
public class Journal extends Registers implements IAnalyzer {
    private String note;
    private String emoji;

    public Journal(Date recordDate, String note, String emoji) {
        super(recordDate);
        this.note = note;
        this.emoji = emoji;
    }

    @Override
    public float calculateScore() {
        return emoji.equals("😊") ? 1f : emoji.equals("😐") ? 0.6f : 0.3f
    }

    @Override
    public String getColorByScore() {
        float s = calculateScore();
        return (s >= 0.8f) ? "Green" : (s >= 0.5f ? "Yellow" : "Red");
    }
}
```

Rationale:
`Journal` adds personal reflection data (`note`, `emoji`) and overrides the analyzer methods to compute an emotional score.
It shows **OCP** and **LSP** in practice: the class extends the system with new analytic behavior without altering existing code, and can substitute other `IAnalyzer` implementations transparently.
This design also supports **ISP**, since it implements only the two required methods from `IAnalyzer`.

```java
public class DailyRecord {
    private List<Registers> registerList = new ArrayList<>();

    public void addRegister(Registers r) { registerList.add(r); }
    public List<Registers> getRegisters() { return registerList; }
}
```

---

4. `DailyRecord` – Collection and Management of Registers

Rationale:

`DailyRecord` is responsible solely for managing a list of records, complying strictly

```java
public class DailyRecord {
    private List<Registers> registerList = new ArrayList<>();

    public void addRegister(Registers r) { registerList.add(r); }
    public List<Registers> getRegisters() { return registerList; }
}
```

with **SRP**.

It depends only on the abstract type `Registers`, not on specific subclasses, which illustrates **DIP**.

This allows any future record type (for instance, `Exercise` or `Hydration`) to integrate seamlessly without code modification, satisfying **OCP**.

---

5. Example of Integration

```java
List<IAnalyzer> habits = Arrays.asList(
    new Sleep(new Date(), 6),
    new Journal(new Date(), "Good day", "😊")
);

ScoreAggregator aggregator = new ScoreAggregator(habits);
System.out.println("Average Score: " + aggregator.getAverageScore());
```

Rationale:
The system uses only the `IAnalyzer` abstraction, not concrete habit classes.
This is a clear demonstration of **DIP** and **LSP**: both `Sleep` and `Journal` behave uniformly as analyzers.
The code is fully extensible—new habits can be introduced through implementation, without altering `ScoreAggregator` or `DailyRecord`.

Overall Effect

These updated class designs collectively illustrate:

- **SRP:** Each class handles one distinct task (record, habit evaluation, data storage).
- **OCP:** New habits or record types can be added via extension, not modification.
- **LSP:** All habits behave consistently through the `IAnalyzer` contract.
- **ISP:** Only necessary methods are implemented in each class.
- **DIP:** Higher-level code interacts with abstractions (`Registers`, `IAnalyzer`), not specific implementations.

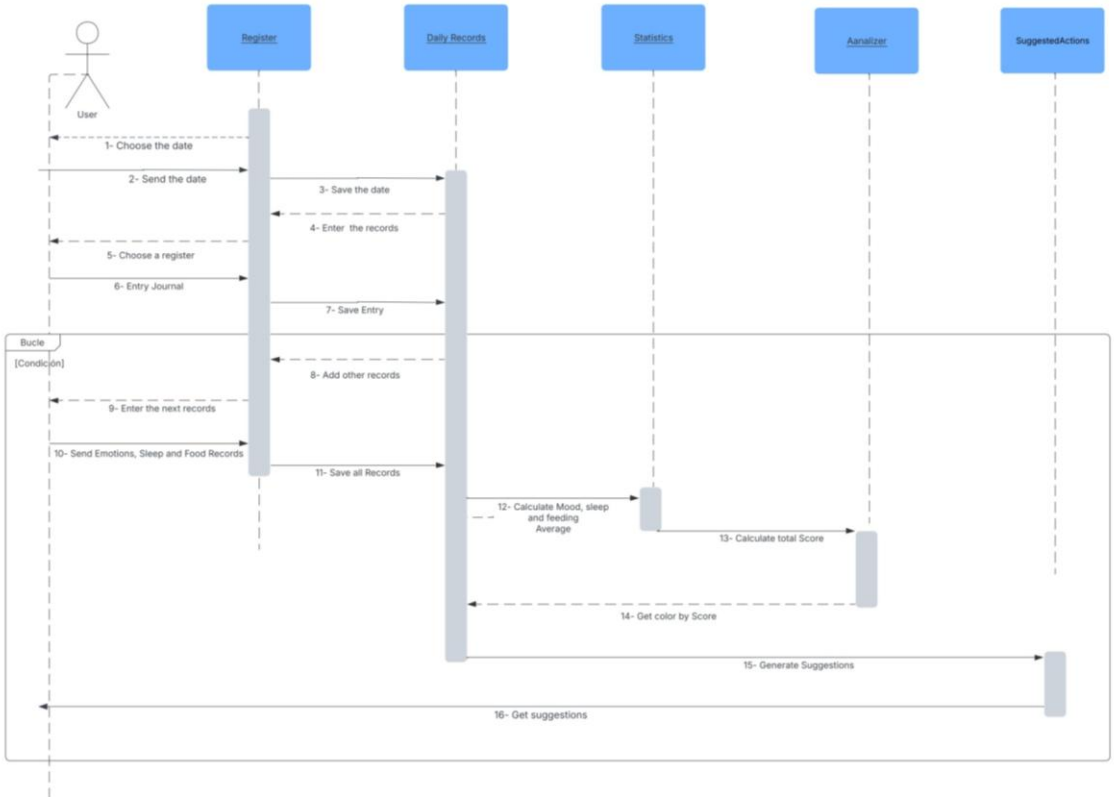
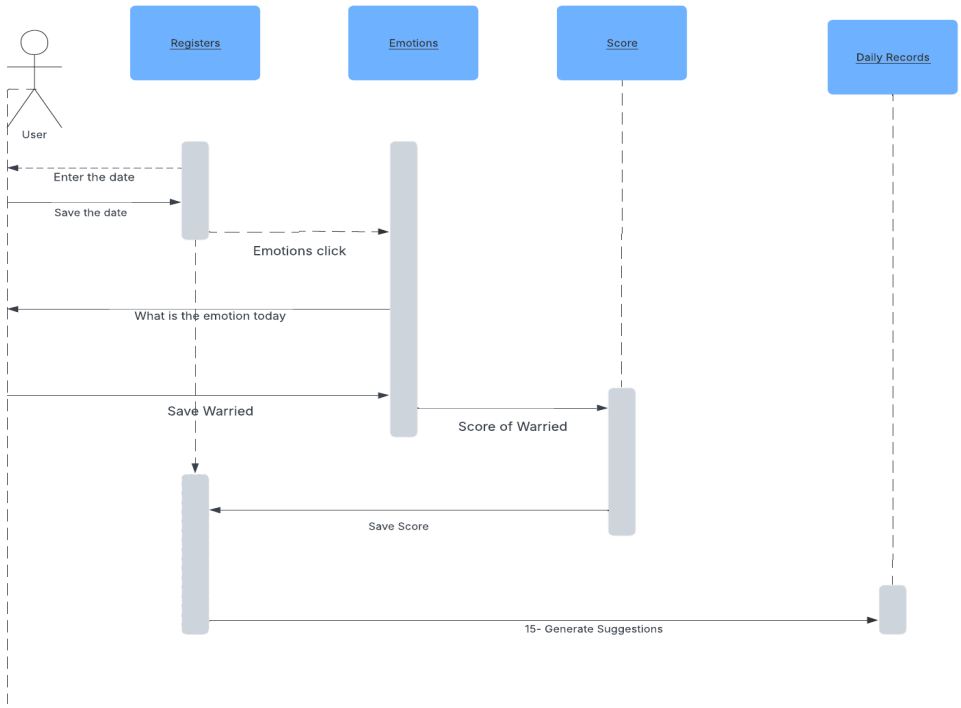
Reflection

Applying the SOLID principles to this project turned out to be both challenging and rewarding. One of the main difficulties was reorganizing the original code, which had several large classes with overlapping purposes. Separating them into smaller, more focused parts without breaking their logic required careful thought and several iterations. Understanding how to apply principles like **Single Responsibility** and **Open/Closed** in a real, functioning system—not just in theory—was a demanding process. It was also challenging to know when to introduce abstraction or interfaces without making the code unnecessarily complex.

Despite those challenges, the refactoring led to major improvements. The code became clearer, easier to modify, and more maintainable. Dependencies were reduced, and each class now has a defined and independent role. Working through these difficulties made the benefits of SOLID much more tangible: the project feels cleaner, more flexible, and closer to real-world software design standards. Overall, the process strengthened both the structure of the code and our understanding of good object-oriented programming practices.

2. Enhanced UML Diagrams (Sequence Diagrams)

General Operation



Record of Emotions, food and sleep